

Figure 6: Data hazard created after ILP

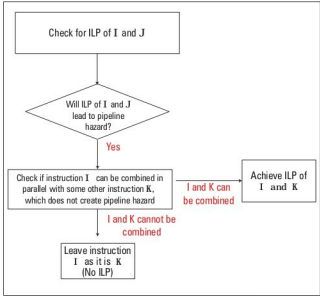


Figure 7: Flowgraph: Dealing with pipeline hazards in case of ILP

The pipeline for the rescheduled instructions would be as shown in Figure 3; we were able to reduce the number of pipeline stalls by one, just by rescheduling the instructions. We might have been able to reduce the pipeline stalls further, if we had had more independent instructions that could be scheduled between *i0* and *i1*.

Voila! Rescheduling instructions is a good method to prevent stalls, but wait! This is not always the case. As a matter of fact, the instruction scheduling itself can sometimes lead to pipeline hazards. Let us see how this happens. To understand this, let's look at an example of instruction scheduling to achieve ILP, and demonstrate its impact on pipeline hazards.

How could ILP lead to a pipeline hazard? Let us suppose we have the set of instructions shown in Figure 4. Now, assume we achieve ILP of Instructions '4' and '8', by

moving Instruction '8' up to the position of Instruction '4', as shown in Figure 5. Notice that Instruction '3' updates the register *R0*, and Instruction '4' uses the register *R0*. Thus, ILP has created a data hazard between Instructions '3' and '4', as shown in Figure 6. The processor will have to insert pipeline stalls in order to prevent the data hazard. (You may refer to my article on *Investigating Pipeline Hazards, LFY*, September 2010, to read more about pipeline hazards, and how they are handled.)

Now, the question is what should be done if ILP can lead to a hazard. One possible solution is to check for the possibility of pipeline hazards, before you achieve ILP. The flowgraph of such an approach would be as shown in Figure 7. Here, we check for the possibility of ILP between two instructions, say I and J. Before achieving ILP, we check if the ILP of I and J will lead to any pipeline hazard. If yes, we check for if it's possible to achieve ILP of Instruction I with some other instruction (K) that does not create any pipeline hazard. If K is found, we achieve ILP of I with K, else we leave Instruction I as it is, without ILP.

However, don't lose heart! This does not mean that we would always lose ILP—we have an alternative. We might be able to achieve ILP despite the pipeline hazard, by modifying our strategy slightly. Now, what you need to do is achieve ILP of I and J, and try to find independent instruction(s) to reschedule, in order to minimise the stalls. (Please refer to the beginning of this article for how to minimise stalls by rescheduling instructions.) Insert the independent instruction(s) in between the instructions causing the pipeline hazard. Thus, your ILP is preserved, and the pipeline hazard is also prevented. 

Glossary	
IF	Instruction Fetch
ID	Instruction Decode
EX	Execute
MA	Memory Access
WB	Write Back
ILP	Instruction Level Parallelism

References and further reading	
1.	Deepti Sharma, 'Exploring the Dynamics of Pipelining', <i>LINUX For You</i> , August 2010
2.	Deepti Sharma, 'Investigating Pipeline Hazards', <i>LINUX For You</i> , September 2010
3.	http://en.wikipedia.org/wiki/Instruction_scheduling

By: Deepti Sharma
The author is a senior software engineer with considerable amount of experience in design, development and quality validation of System Software Tools (compilers/assemblers/device drivers) for embedded systems. She has also been mentor/trainer for training on the compiler development, following good practices and maintaining quality. She loves writing for LFY. She can be reached at deepti.acmet@gmail.com .